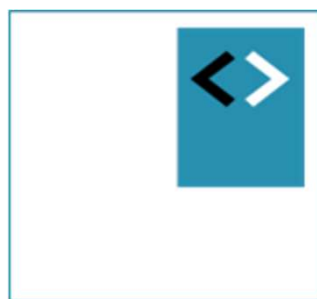




TBWB

Angular Fundamentals

Module 3 - Services

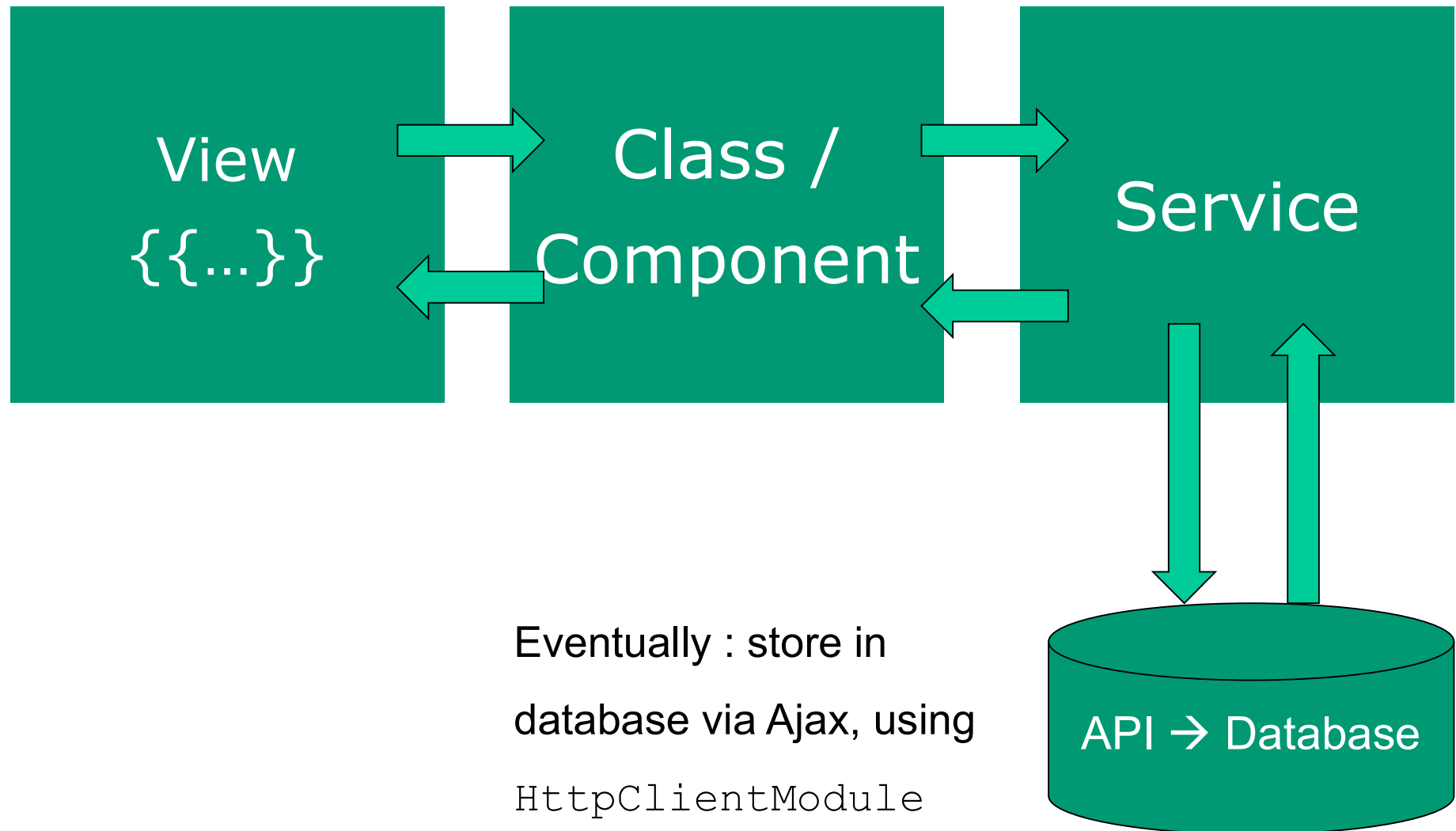


Peter Kassenaar –
info@kassenaar.com

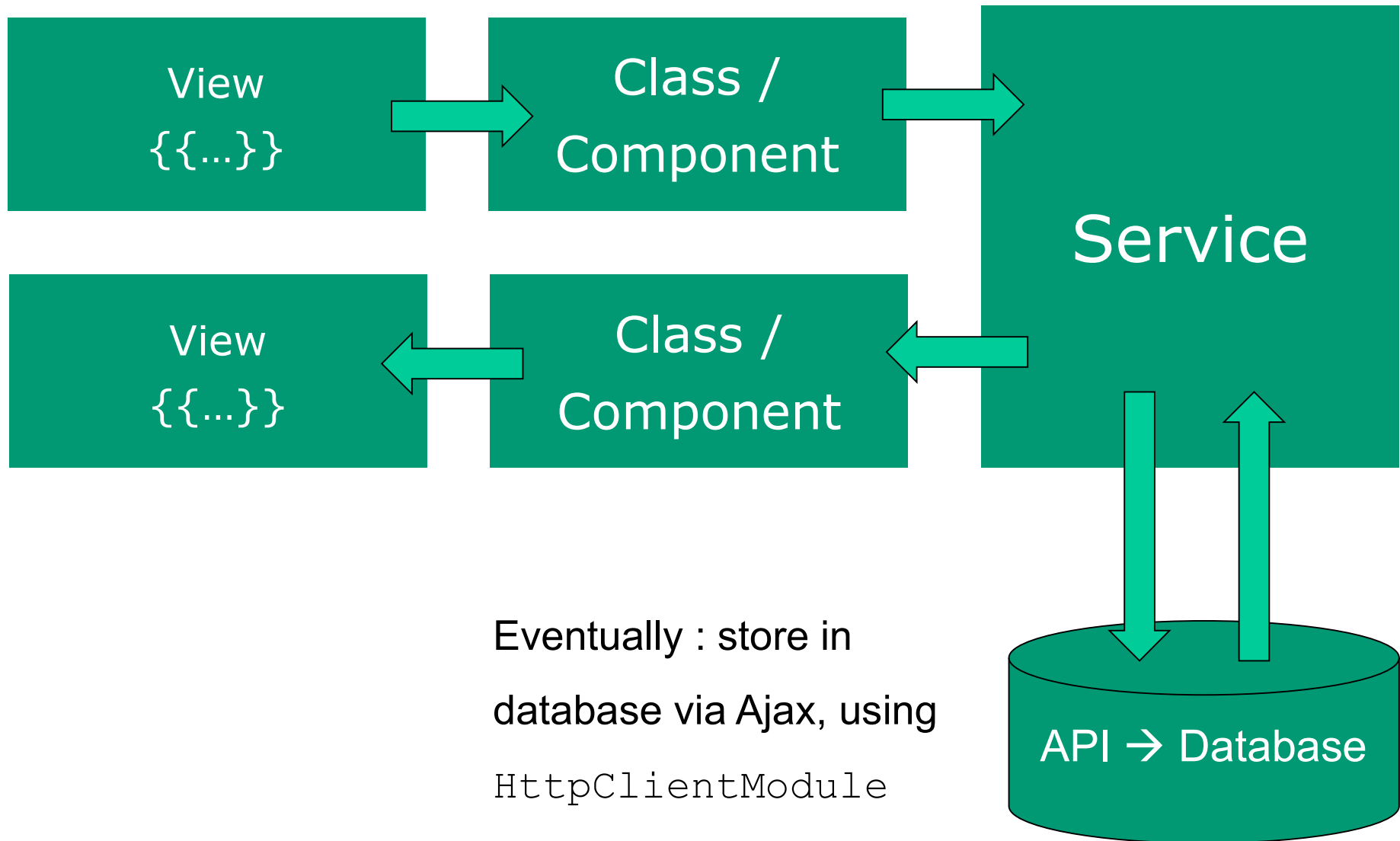
Services

- Goal – *reuse* data functionality over different components
 - Data retrieval
 - Data caching
 - Data Storage,
 - ...
- Angular: create a class without a UI (no template)
 - `export class myDataService { ... }`

Data flow



...and with multiple components



Services in Angular

Data services in Angular 1.x:

```
angular.module('myApp')  
  .service(...)  
  .factory(...)  
  .provider(...)
```

Data services in Angular:

```
import {Injectable} from '@angular/core';  
  
@Injectable({providedIn: 'root'})  
export class CityService{  
  //....  
}
```

Official documentation on services/DI

← Essentials

Overview

Composition with components

Reactivity with signals

Dynamic interfaces with templates

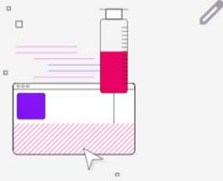
Modular design with dependency injection

Next Steps

Introduction > Essentials

Dependency Injection

Reuse code and control behaviors across your application and tests.



When you need to share logic between components, Angular leverages the design pattern of [dependency injection](#) that allows you to create a "service" which allows you to inject code into components while managing it from a single source of truth.

What are services?

Services are reusable pieces of code that can be injected

Similar to defining a component, services are comprised of the following:

- A **TypeScript decorator** that declares the class as an Angular service via `@Injectable` and allows you to define what part of the application can access the service via the `providedIn` property (which is typically `'root'`) to allow a service to be accessed anywhere within the application.
- A **TypeScript class** that defines the desired code that will be accessible when the service is injected

Here is an example of a `Calculator` service.

```
import {Injectable} from '@angular/core';

@Injectable({providedIn: 'root'})
export class Calculator {
```

<https://angular.dev/essentials/dependency-injection>

Make sure to use @Injectable

Why? – Dependency Injection (DI) en metadata!

*"TypeScript sees the @Injectable() decorator and emits metadata about our service, metadata that Angular may need to **inject** other dependencies into this service."*

<https://angular.io/docs/ts/latest/tutorial/toh-pt4.html>

But...

"Our service doesn't have any dependencies at the moment". Add the decorator anyway.

*It is a best practice to apply the
@Injectable() decorator from the start both
for consistency and for future-proofing"*



Creating a service

Creating a service in 3 steps

Creating a service – 2 steps

1. **Create**/generate your service
2. Consume/**inject** service into component
3. Classic: Make service available in the module
 - NOT necessary when using modern notation (see further)

Creating a service

```
ng generate service [name]
```

Step 1 – create service (currently: static data)

```
import { Injectable, signal } from '@angular/core';
import { City } from '../model/city.model';

// Injectable decorator marks it as a service that can be injected
@Injectable({
  providedIn: 'root'
})
export class CityService {
  // A private signal that holds the array of cities.
  private cities = signal<City[]>([
    new City(1, 'Groningen', 'Groningen'),
    new City(2, 'Hengelo', 'Overijssel'),
    new City(3, 'Den Haag', 'Zuid-Holland'),
    new City(4, 'Enschede', 'Overijssel'),
  ]);

  // Method to get all cities as a readonly signal
  getCities() {
    return this.cities.asReadonly();
  }
  // etc...
}
```

Step 2 – Inject Service to be 'consumed'

Dependency Injection

Using the inject() function

```
export class App {  
  // Inject the CityService  
  private cityService = inject(CityService);  
  
  // Signals for cities, current city, and city photo  
  protected cities = this.cityService.getCities();  
  protected currentCity = signal<City | null>(null);  
  protected cityPhoto = signal<string>('');  
  
  // Method to update the current city and photo  
  updateCity(city: City) {  
    this.currentCity.set(city);  
    const imageFilename = `assets/img/${city.name}.jpg`;   
    this.cityPhoto.set(imageFilename);  
  }  
}
```

Setting the signal value

Classic – using constructor () injection

```
...
import {CityService} from "../city.service";

@Component({
  selector    : 'hello-world',
  templateUrl: 'app/app.component.html',
})

export class AppComponent implements OnInit {
  // Properties for component/class
  currentCity: City;
  cities: City[];
  cityPhoto: string;

  constructor(private cityService: CityService) {
  }

  ngOnInit() {
    this.cities = this.cityService.getCities();
  }
}
```

Constructor Injection

Call service method in
ngOnInit()

Want to learn more on DI? Check docs

How does dependency injection work in Angular?

A dependency is any object, value, function or service that a class needs to work but does not create itself. In other words, it creates a relationship between different parts of your application since it wouldn't work without the dependency.

There are two ways that code interacts with any dependency injection system:

- Code can *provide*, or make available, values.
- Code can *inject*, or ask for, those values as dependencies.

"Values," in this context, can be any JavaScript value, including objects and functions. Common types of injected dependencies include:

- **Configuration values:** Environment-specific constants, API URLs, feature flags, etc.
- **Factories:** Functions that create objects or values based on runtime conditions
- **Services:** Classes that provide common functionality, business logic, or state

Angular components and directives automatically participate in DI, meaning that they can inject dependencies *and* they are available to be injected.

What are services?

An Angular *service* is a TypeScript class decorated with `@Injectable`, which makes an instance of the class available to be injected as a dependency. Services are the most common way of sharing data and functionality across an application.

<https://angular.dev/guide/di>

In Classic Angular - Instantiation?

- Pay attention: no manual `new()` instance of Service!

- Services are -often- Singletons
- Are fetched from the Module and/or instantiated in

`constructor()`

```
constructor(private cityService:CityService) { ... }
```

"The constructor itself does nothing.

*The parameter simultaneously defines a
private cityService property and identifies
it as a CityService injection service."*

Traditional – constructor based DI

- Not “wrong”! This is still valid in Angular 17+

```
export class InjectComponent {  
  
    // using classic constructor based Dependency Injection:  
    constructor(public userService: UserService) {  
    }  
  
    //...  
}
```

Modern Angular (17+) the `inject()` function

- Technically available since Angular 14
 - but now a lot more useful!
 - Ditching *Constructor based Dependency Injection*
- Technical benefits
 - Readability benefits – a lot
 - It's a lot more clearer what dependencies a component needs – even if you don't fully understand DI.
 - Inheritance becomes much simpler. No need to use `super(service1, service2) ... on extends BaseComponent`

```
export class InjectComponent {  
  // using inject() based Dependency Injection  
  userService = inject(UserService);  
}
```

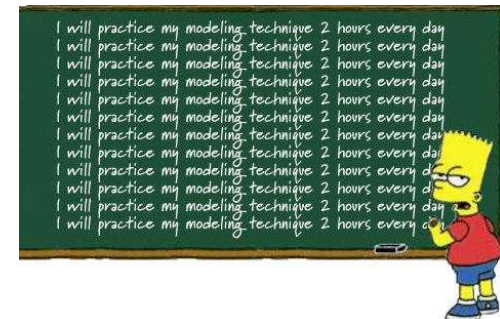
Warning! Inject only during constructor-time

- Note: the `inject()` function only works when constructing the component
- So you CAN NOT use `inject()` in other methods

```
export class InjectComponent {  
  
    // works - executed in construction phase of component  
    userService = inject(UserService);  
  
    ngOnInit(){  
        // won't work, this function is executed later  
        inject(SomeService).someValue  
    }  
    someMethod(){  
        // won't work, this function is executed later  
        inject(SomeService).someValue  
    }  
}
```

Workshop

- While working, you have developed some components, displaying data. **Move this data** from the component class to a service.
- **Create a new file** (for example `city.service.ts`) and import the correct annotations. Use `@Injectable()` to decorate the service.
 - options: use constructor DI, or `inject()`.
- Write a `getCities()` method and **methods to add or delete**
 - or the data you are working with, of course.
- Example: `\200-services-static`



Checkpoint

- Every service in Angular is a `class`
- Use the `@Injectable()` decorator on service classes
- Modern: use **Dependency Injection** with `inject()` in the component that consumes the service
 - The `Injector` makes sure all components get the same instance
 - No manual `new XyxService()` in your code.
- Classic: Dependency Inject in `constructor()` of the component that consumes/needs access to the service methods